Engaging GPU Howto Recipes PyTorch

Deep Learning with PyTorch on GPUs

Deep learning is the foundation of artificial intelligence nowadays. Deep learning programs can be accelerated substantially on GPUs.

PyTorch is a popular Python package for working on deep learning projects.

This page introduces recipes to run deep-learning programs on GPUs with PyTorch.

Installing PyTorch

First, load a Miniforge module to provide a Python platform with PyTorch preinstalled with CUDA support, which enables running on GPUs,

```
module load miniforge/24.3.0-0
```

PyTorch on CPU and a single GPU

We start with a recipe to run PyTorch on one CPU and one GPU.

We use an example code training a convolutional neural network (CNN) with the CIFAR10 data set. Refer to the description of this example and download the codes for CPU and for GPU.

Prepare a job script named `job.sh` like this,

```
#!/bin/bash
#SBATCH -p mit_normal_gpu
#SBATCH --gres=gpu:1
#SBATCH -t 30
#SBATCH -N 1
#SBATCH -n 2
#SBATCH --mem=10GB

module load miniforge/24.3.0-0

echo "~~~~~~~~ Run the program on CPU ~~~~~~~~~"
time python cnn_cifar10_cpu.py
echo "~~~~~~~~ Run the program on GPU ~~~~~~~~~"
```

```
time python cnn_cifar10_gpu.py
```

then submit it,

```
sbatch job.sh
```

The `mit_normal_gpu` partition is for all MIT users. If your lab has a partition with GPUs, you can use it too.

The `#SBATCH` flags `-N 1 -n 2` requests two CPU cores on one node, and the `--mem=10GB` means 10 GB of memory per node (not per core).

The programs `cnn_cifar10_cpu.py` and `cnn_cifar10_gpu.py` will run on CPUs and a GPU, respectively. When the problem size is large, the program will be accelerated on a GPU.

While the job is running, you can check if the program runs on a GPU. First, check the hostname that it runs on,

```
squeue --me
```

and then log in the node,

```
ssh <nodeXXX>
```

and check the GPU usage with the `nvtop` command. Documentation for `nvtop` can be found here.

PyTorch on multiple GPUs

Deep learning programs can be further accelerated on multiple GPUs.

There are various parallelisms to enable distributed deep learning on multiple GPUs, including data parallel and model parallel. We will focus on data parallel on this page.

Data parallel allows training a model with multiple batches of data simultaneously. The model has to fit into the GPU memory.

On a cluster, there are many nodes and multiple GPUs on each node. We will first introduce a recipe to run PyTorch programs with multiple GPUs within one node, and then extend it to multiple nodes.

We use an example code that trains a linear network with a random data set, which is implemented with the Distributed Data Parallel package in PyTorch. Refer to the description of this example for multiple GPUs within one node and for multiple GPUs across multiple nodes.

Download the codes for this example: datautils.py, multigpu.py, multigpu_torchrun.py, and multinode.py.

Single-node multi-GPU data parallel

In this section, we introduce a recipe for single-node multi-GPU data parallel. The program `multigpu.py` is set up for this purpose.

To run the program on 2 GPUs within one node, prepare a job script named `job.sh` like this,

```
#!/bin/bash
#SBATCH -p mit_normal_gpu
#SBATCH --job-name=ddp
#SBATCH -N 1
#SBATCH -n 2
#SBATCH --mem=20GB
#SBATCH --gres=gpu:2

module load miniforge/24.3.0-0

echo "======== Run on multiple GPUs ========"
# run for 100 epochs and save checkpoints every 20 epochs
python multigpu.py --batch_size=1024 100 20
```

then submit it,

```
sbatch job.sh
```

The `-N 1 -n 2 --gres=gpu:2` flags request 2 CPU cores and 2 GPUs on one node. For most GPU programs, it is recommended to set the number of CPU cores no less than the number of GPUs.

As is set up in the code `multigpu.py`, it will run on all of the GPUs requested in Slurm, which means 2 GPUs within one node in this case. The training process happens on 2 batches of data simultaneously.

You can try to check if the program runs on multiple GPUs using the `nvtop` command as described in the previous section, but the program runs so fast that this might be challenging. You can also check in the Slurm out file that GPUs with different IDs were utilized.

There is another way to run a PyTorch prgram with multiple GPUs, that is to use the `torchrun` command. The program for this purpose is `multigpu_torchrun.py`. In the above job script, change the last line to this,

```
torchrun --nnodes=1 --nproc_per_node=2 \
         --rdzv_id=$SLURM_JOB_ID \
```

```
        --rdzv_endpoint="localhost:1234" \
        multigpu_torchrun.py --batch_size=1024 100 20
```

With the flags `--nnodes=1 --nproc-per-node=2`, the `torchrun` command will run the program on 2 GPUs within one node.

The flags with `rdzv` (meaning the Rendezvous protocol) are required by `torchrun` to coordinate multiple processes. The flag `--rdzv-id=$SLURM_JOB_ID` sets to the `rdzv` ID be the job ID, but it can be any random number. The flag `--rdzv-endpoint=localhost:1234` is to set the host and the port. Use `localhost` when there is only one node. The port can be any 4- or 5-digit number larger than 1024.

The `torchrun` command will be useful for running the program across multiple nodes in the next section.

GPU communication within one node

The NVIDIA Collective Communications Library (NCCL) is set as the backend in the PyTorch programs `multigpu.py` and `multigpu_torchrun.py`, so that the data communication between GPUs within one node benefits from the high bandwidth of NVLinks.

Multi-node multi-GPU data parallel

Now we extend the above example to multi-node multi-GPU data parallel. The program `multinode.py` is set up for this purpose.

There are two key points in this approach.

1. Use the `srun` command in Slurm to launch a `torchrun` command on each node.
2. Set up `torchrun` to coordinate processes on different nodes.

To run on multiple GPUs across multiple nodes, prepare a job script like this,

```
#!/bin/bash
#SBATCH -p mit_normal_gpu
#SBATCH --job-name=ddp-2nodes
#SBATCH -N 2
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=2
#SBATCH --gpus-per-node=2
#SBATCH --mem=20GB

module load miniforge/24.3.0-0
```

```
# Get IP address of the master node
nodes=( $( scontrol show hostnames $SLURM_JOB_NODELIST ) )
nodes_array=($nodes)
master_node=${nodes_array[0]}
master_node_ip=$(srun --nodes=1 --ntasks=1 -w "$master_node" hostname --ip-
address)

echo "======== Run on multiple GPUs across multiple nodes ======"
srun torchrun --nnodes=$SLURM_NNODES \
     --nproc-per-node=$SLURM_CPUS_PER_TASK \
     --rdzv-id=$SLURM_JOB_ID   \
     --rdzv-backend=c10d \
     --rdzv-endpoint=$master_node_ip:1234 \
     multinode.py --batch_size=1024 100 20
```

then submit it,

```
sbatch job.sh
```

As the `#SBATCH` flags `-N 2` and `--ntasks-per-node=1` request for two nodes with one task per node, the `srun` command launches a `torchrun` command on each of the two nodes.

The `#SBATCH` flags `--cpus-per-task=2` and `--gpus-per-node=2` request 2 GPU cores and 2 GPUs on each node. Accordingly, the `torchrun` flags are set as `--nnodes=$SLURM_NNODES --nproc-per-node=$SLURM_CPUS_PER_TASK`, so that the `torchrun` command runs the program on 2 GPUs on each of the two nodes. That says the program runs on 4 GPUs, and thus the training process happens on 4 batches of data simultaneously.

The flags with `rdzv` are required by `torchrun` to coordinate processes across nodes. The `--rdzv-backend=c10d` is to use a C10d store (by default TCPStore) as the rendezvous backend, the advantage of which is that it requires no 3rd-party dependency. The `--rdzv-endpoint=$master_node_ip:1234` is to set up the IP address and the port of the master node. The IP address is obtained in a previous part of the job script.

Refer to details of torchrun on this page.

GPU communication across nodes

The NCCL is set as backend in the PyTorch program `multinode.py`, so that the data communication between GPUs within one node benefits from the high bandwidth of NVLinks, and the data communication between GPUs across nodes benefits from the bandwidth of the Infiniband network.

February 11, 2026